Deakin University

Object Oriented Development

OnTrack Submission

C# Essentials: Classes and Objects

Submitted By:
Yuchen Qiu
s225395576
2026/03/25 16:59

Tutor:
Sohaib Aslam

March 25, 2026

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```
class MobileProgram
{
    static void Main(string[] args)
    {
        // First mobile
        Mobile m1 = new Mobile("Monthly", "Samsung", "12345");

        Console.WriteLine("First Mobile:");
        Console.WriteLine(m1.getAccType());
        Console.WriteLine(m1.getDevice());
        Console.WriteLine(m1.getNumber());
        Console.WriteLine(m1.getBalance());

        Console.WriteLine();

        m1.addCredit(10);
        m1.makeCall(5);
        m1.sendText(2);

        Console.WriteLine();

        // Second mobile
        Mobile m2 = new Mobile("PAYG", "iPhone", "67890");

        Console.WriteLine("Second Mobile:");
        Console.WriteLine(m2.getAccType());
        Console.WriteLine(m2.getDevice());
        Console.WriteLine(m2.getNumber());
        Console.WriteLine(m2.getBalance());

        Console.WriteLine();

        m2.addCredit(20);
        m2.makeCall(3);
        m2.sendText(1);

        Console.ReadLine();
    }
}
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```
class Mobile
{
    private string accType;
    private string device;
    private string number;
    private double balance;

    private const double CALL_COST = 0.245;
    private const double TEXT_COST = 0.078;

    public Mobile(string accType, string device, string number)
    {
        this.accType = accType;
        this.device = device;
        this.number = number;
        this.balance = 0.0;
    }

    public string getAccType()
    {
        return accType;
    }

    public string getDevice()
    {
        return device;
    }

    public string getNumber()
    {
        return number;
    }

    public string getBalance()
    {
        return balance.ToString("C");
    }

    public void setAccType(string accType)
    {
        this.accType = accType;
    }

    public void setDevice(string device)
    {
        this.device = device;
    }

    public void setNumber(string number)
    {
```

```
        this.number = number;
    }

    public void setBalance(double balance)
    {
        this.balance = balance;
    }

    public void addCredit(double amount)
    {
        balance += amount;
        Console.WriteLine("Credit added. Balance: " + getBalance());
    }

    public void makeCall(int minutes)
    {
        balance -= minutes * CALL_COST;
        Console.WriteLine("Call made. Balance: " + getBalance());
    }

    public void sendText(int numTexts)
    {
        balance -= numTexts * TEXT_COST;
        Console.WriteLine("Text sent. Balance: " + getBalance());
    }
}
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```
class EmployeeProgram
{
    static void Main(string[] args)
    {
        // Create employee
        Employee e1 = new Employee("John", 50000);

        Console.WriteLine("Employee Name: " + e1.getName());
        Console.WriteLine("Salary: " + e1.getSalary());

        Console.WriteLine();

        // Raise salary
        e1.raiseSalary(10);

        Console.WriteLine("After raise:");
        Console.WriteLine("Salary: " + e1.getSalary());

        Console.WriteLine();

        // Tax
        Console.WriteLine("Tax: $" + e1.Tax());

        Console.ReadLine();
    }
}
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```
class Employee
{
    private string name;
    private double salary;

    // Constructor
    public Employee(string name, double salary)
    {
        this.name = name;
        this.salary = salary;
    }

    // Getter
    public string getName()
    {
        return name;
    }

    public string getSalary()
    {
        return salary.ToString("C", CultureInfo.GetCultureInfo("en-AU"));
    }

    // Raise salary
    public void raiseSalary(double percent)
    {
        salary += salary * percent / 100;
    }

    // Tax calculation
    public double Tax()
    {
        if (salary <= 18200)
        {
            return 0;
        }
        else if (salary <= 37000)
        {
            return (salary - 18200) * 0.19;
        }
        else if (salary <= 90000)
        {
            return 3572 + (salary - 37000) * 0.325;
        }
        else if (salary <= 180000)
        {
            return 20797 + (salary - 90000) * 0.37;
        }
        else
        {
```

```
            return 54096 + (salary - 180000) * 0.45;
        }
    }
}
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```
using System;

class CarProgram
{
    static void Main(string[] args)
    {
        Car car1 = new Car(25);

        Console.WriteLine("Fuel cost per litre: " + car1.printFuelCost());
        Console.WriteLine("Initial fuel: " + car1.getFuel() + " litres");
        Console.WriteLine("Initial total miles: " + car1.getTotalMiles());

        Console.WriteLine();

        car1.addFuel(40);

        Console.WriteLine();
        Console.WriteLine("Fuel in tank: " + car1.getFuel().ToString("F2") + "
        ↪  litres");

        Console.WriteLine();

        car1.drive(50);

        Console.WriteLine();
        Console.WriteLine("Fuel left: " + car1.getFuel().ToString("F2") + "
        ↪  litres");
        Console.WriteLine("Total miles driven: " + car1.getTotalMiles());

        Console.ReadLine();
    }
}
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The orginal submission can be retrieved via the "Download Uploaded Files" function.

```
using System;
using System.Globalization;
class Car
{
    private double mpg;
    private double fuel;
    private double totalMiles;

    private const double FUEL_COST = 1.385;

    // Constructor
    public Car(double mpg)
    {
        this.mpg = mpg;
        this.fuel = 0;
        this.totalMiles = 0;
    }

    // Accessor methods
    public double getFuel()
    {
        return fuel;
    }

    public double getTotalMiles()
    {
        return totalMiles;
    }

    // Mutator method
    public void setTotalMiles(double totalMiles)
    {
        this.totalMiles = totalMiles;
    }

    // Return fuel cost as currency
    public string printFuelCost()
    {
        return FUEL_COST.ToString("C", CultureInfo.GetCultureInfo("en-AU"));
    }

    // Add fuel and display cost
    public void addFuel(double litres)
    {
        fuel += litres;
        Console.WriteLine("Added fuel: " + litres + " litres");
        Console.WriteLine("Cost of fill: " + calcCost(litres).ToString("C",
        ↪  CultureInfo.GetCultureInfo("en-AU")));
    }

```

```
    // Calculate cost of fuel
    public double calcCost(double litres)
    {
        return litres * FUEL_COST;
    }

    // Convert gallons to litres
    public double convertToLitres(double gallons)
    {
        return gallons * 4.546;
    }

    // Drive the car
    public void drive(double milesDriven)
    {
        double gallonsUsed = milesDriven / mpg;
        double litresUsed = convertToLitres(gallonsUsed);

        fuel -= litresUsed;
        totalMiles += milesDriven;

        Console.WriteLine("Drove " + milesDriven + " miles");
        Console.WriteLine("Fuel used: " + litresUsed.ToString("F2") + " litres");
        Console.WriteLine("Cost of journey: " + calcCost(litresUsed).ToString("C",
        ↪ CultureInfo.GetCultureInfo("en-AU")));
    }
}
```